```
    var rectNumber = d3.select("#rightSVG")
         .selectAll("rect").size();
    var rectData = d3.select("#rightSVG").selectAll("rect").data();
    var rectMax = d3.max(rectData, function(d) {return d.values.length});

    var rightSize = canvasSize("#rightSVG");

barXScale = d3.scale.ordinal()
      .domain(rectData.map(function(d){return d.key}))
      .rangeBands([0, rightSize[0]]);

barYScale = d3.scale.linear()
   .domain([0, rectMax])
   .range([rightSize[1],0]);

d3.select("#rightSVG")
   .selectAll("rect")
   .attr("x", function(d,i) {return barXScale(d.key) + 5})
   .attr("width", function() {return barXScale.rangeBand() - 5})
   .attr("y", function(d) {return barYScale(d.values.length)})
   .attr("height", function(d) {
        return rightSize[1] - barYScale(d.values.length);
   });

function canvasSize(targetElement) {
   var height = parseFloat(d3.select(targetElement)
                     .node().clientHeight);
 var width = parseFloat(d3.select(targetElement).node().clientWidth);
 return [width,height];
};

};
```

We need to calculate a scale dynamically based on the bar chart data to resize it.

x scale from an ordinal scale based on the "key" value created by d3.nest

y scale from the maximum length of the "values" array created by d3.nest

Width and x position with a 5-pixel margin

Calculate the current size of the SVG element in pixels

We use the `rangeBands` function of `d3.scale.ordinal()` to efficiently create a reusable bar chart. `rangeBands` divides the given width into equal areas, which is useful for a bar chart. You may be tempted to use the bare object array as your domain, but `rangeBands` uses the string version of the domain objects, which results in `"[object object]"` rather than unique identifiers for each object. Instead, we'll create a mapping of our array based on the key values (the names of each tweet creator) to use as our domain. For the range array of the ordinal scale, we'll use the current width of the `svg` element where it's being used, which we calculate using `canvasSize()`.

Our redraw function finally produces graphics for a screenshot. With this code, the charts respond to changes in screen size, as demonstrated by the various versions in figure 9.4.

I didn't make any redraw functions for the spreadsheet, because it's a more involved process that follows the same principles. But now that we have a dashboard that loads the data into different views and resizes automatically, it's time to wire the views together to make it interactive.